

AI 私有知识库与 RAG 项目实施技术文档

一、项目背景介绍

随着各行各业数字化转型持续推进，企业内部沉淀了大量项目方案、操作手册、运维文档、业务规范、售后资料等非结构化文档。传统文件夹检索、关键词搜索方式效率低下，员工想要查阅业务资料需要花费大量时间翻找文件，同时资料版本杂乱、分散存储，极易出现信息不一致、文档丢失等问题。

大语言模型具备强大的自然语言理解能力，但原生大模型存在知识截止、容易产生信息幻觉，无法直接读取企业内部私有资料。检索增强生成技术

（RAG, Retrieval-Augmented Generation）很好解决了该痛点。RAG 架构可以将企业本地私有文档转化为向量数据存入向量数据库，用户发起提问时，系统先从知识库检索和问题高度相关的文档片段，将检索结果作为上下文交给大模型，让模型依托真实文档内容作答，大幅减少凭空编造内容的情况，实现企业内部资料智能问答。

本项目搭建轻量化本地 RAG 问答系统，无需依赖复杂云平台，在单机环境完成文档解析、文本预处理、向量入库、相似度检索、智能问答全流程，可作为信息化实施项目原型，后续能够扩展部署到企业内网，用于项目实施知识库、售后支持资料查询、系统操作手册智能检索等场景。

二、RAG 系统完整工作流程

一套标准 RAG 系统分为两大阶段：文档预处理入库阶段、线上问答推理阶段。

2.1 文档预处理入库阶段

第一步：文档加载。支持 PDF、TXT、Markdown 等常见格式文档读取，通过文档加载器提取文档内部文本内容，过滤图片、水印、无效页眉页脚等无关信息。在项目实施场景中，需要导入的文档包含实施方案、部署手册、故障排查指南、用户培训资料等多种类型文件。

第二步：文本清洗。原始文档经常存在多余空行、连续空格、特殊符号、换行错乱、重复段落。清洗环节需要统一文本格式，剔除无意义字符，合并碎片化短句，删除重复内容，保证文本质量。低质量文本会直接降低后续检索准

确度，也是数据治理环节重点工作。

第三步：文本切分（文本分块）。大模型存在上下文长度限制，不能一次性传入完整长篇文档。需要采用文本分割器将长文档切分成固定长度文本块。分块参数需要合理调试：块长度设置过大，单段文本信息繁杂，检索精准度下降；块长度设置过小，文本丢失上下文逻辑，答案完整性不足。工程实践中通常会设置块重叠，保障段落交界处信息不会被切断。

第四步：文本向量化（Embedding）。使用嵌入模型将人类可读的文本转化为多维浮点向量。语义含义相近的文本，对应的向量在向量空间距离更近。本项目采用开源 BGE 向量模型，支持中文场景，在私有知识库任务上表现均衡，并且可以本地离线运行，不需要持续调用第三方接口。

第五步：向量持久化存储。生成完成的向量和原始文本片段一一对应，存入向量数据库。向量数据库针对向量相似度检索做了专项优化，区别于传统关系型 MySQL 数据库。本项目使用 Chroma 轻量级向量库，部署简单、开箱即用，非常适合中小型知识库原型搭建与项目验证。

2.2 线上问答推理阶段

第一步：用户输入问题。系统接收用户自然语言提问。

第二步：问题向量化。使用相同的 Embedding 模型，把用户问题转化为查询向量。

第三步：相似度检索。在向量库中计算查询向量与存量文档向量之间距离，召回相似度排名靠前的若干条文本片段。

第四步：Prompt 组装。把原始问题、检索到的参考文档片段，按照预设模板拼接，形成完整提示词送入大模型。同时增加约束指令，要求模型优先依托参考文档作答，文档没有相关信息时如实告知，禁止随意编造内容。

第五步：大模型生成回答。大模型接收组装后的 Prompt，输出贴合企业内部资料的答案，返回给前端展示。

三、程序模块设计说明

本项目采用面向对象方式封装 RAGSystem 类，代码结构清晰，便于维护、扩展和二次开发，符合工程化编码规范。具体模块划分如下：

1. 文档解析模块：基于 PyPDF 库实现 PDF 文档文本提取，可后续拓展 Word、Excel 文档解析能力。

2. 文本处理模块：集成递归字符文本分割器，支持自定义块大小、重叠长度参数。

3. 向量库管理模块：封装向量库新建、持久化、检索方法。

4. LLM 交互模块：统一封装大模型接口调用，切换不同厂商大模型时仅修改少量配置。

5. 问答链路整合模块：串联检索与生成流程，形成端到端问答能力。

在实际项目实施场景里，模块化代码便于实施工程师根据客户需求调整功能。例如部分客户要求限定文档检索范围、增加问答日志记录、导出问答记录，都可以在现有类结构上快速扩展开发。

四、部署与实施注意事项

4.1 本地运行环境

推荐 Python3.9 ~ Python3.11 版本，创建独立虚拟环境管理依赖包，避免不同项目包版本冲突。项目依赖包含 LangChain 生态组件、文档解析库、向量库、嵌入模型相关工具。部署前通过 requirements.txt 统一安装依赖，方便环境快速复刻。

4.2 文档准备规范

1. 优先使用文字版 PDF，扫描件图片 PDF 无法直接提取文本，需要额外接入 OCR 工具；
2. 提前梳理文档目录结构，按照业务模块分类导入知识库，方便后期管理；
3. 定期对知识库进行巡检，删除过时方案、作废版本文档，持续维护数据质量。

4.3 上线拓展方向

原型验证完成后，可以进行工程化升级：增加 Web 前端页面、接入用户权限管理、增加问答日志统计、对接企业已有统一登录平台。同时支持私有化部署在内网服务器，满足部分企业数据不出网的安全要求。

五、落地业务场景

场景 1：信息化项目实施知识库。实施工程师在外驻场时，可以快速检索系统部署步骤、常见报错解决方案、配置参数规范，减少翻阅大量文档的时间，提升现场调试效率。

场景 2：售后技术支持。客服人员检索故障处理文档，标准化回复客户问

题，保障不同人员给出的解决方案统一规范。

场景 3：新人培训资料问答。新入职员工可以通过自然语言提问快速查阅操作手册，降低培训成本。

场景 4：项目方案检索。商务、实施人员快速调取历史同类项目方案要点，辅助新项目方案撰写。

六、总结与拓展思考

RAG 是当前大模型落地企业场景主流方案，门槛适中、风险可控，广泛应用于各类信息化项目。作为实施人员，不仅需要掌握系统部署运行，还要理解底层数据处理逻辑，能够和客户沟通需求、排查检索异常、持续优化知识库质量。

后续持续优化方向：优化文档切片策略、引入重排序模型提升检索精度、增加多轮对话记忆、支持图文多模态文档解析、接入监控看板统计知识库访问情况。在项目交付阶段，可以配套输出部署手册、运维文档、操作培训材料，形成完整交付物，满足政企项目标准化实施交付要求。